**Universidade da Amazônia (UNAMA)**
Curso: Ciência da Computação

**ALAN DOS ANJOS LEÃO**
**LUCAS SOARES VILHENA**

# DEVOPS NA PRÁTICA: PIPELINE CI/CD, KUBERNETES COM HELM E MONITORAMENTO COM ELASTIC STACK

Belém – PA
2026

**ALAN DOS ANJOS LEÃO**
**LUCAS SOARES VILHENA**

# DEVOPS NA PRÁTICA: PIPELINE CI/CD, KUBERNETES COM HELM E MONITORAMENTO COM ELASTIC STACK

Relatório da Atividade Final apresentado à disciplina DevOps Tools, do curso de Ciência da Computação, da Universidade da Amazônia (UNAMA), como requisito de avaliação.

Professor: Rodrigo Marques

Belém – PA
2026

**SUMÁRIO**

1 INTRODUÇÃO ..... 4
2 FUNDAMENTAÇÃO TEÓRICA ..... 4
2.1 DevOps e sua relação com as metodologias ágeis ..... 4
2.2 Integração contínua, entrega contínua e implantação contínua ..... 5
2.3 Máquinas virtuais e containers ..... 5
3 METODOLOGIA E FERRAMENTAS ..... 6
4 DESENVOLVIMENTO ..... 7
4.1 Aplicação ..... 7
4.2 Integração e entrega contínuas (Etapa 2) ..... 7
4.3 Implantação contínua e orquestração (Etapa 3) ..... 9
4.4 Logging e métricas (Etapa 4) ..... 10
5 RESULTADOS E DESAFIOS ..... 11
6 CONSIDERAÇÕES FINAIS ..... 12
REFERÊNCIAS ..... 13

# 1 INTRODUÇÃO

A entrega de software em ciclos curtos e com qualidade exige que desenvolvimento e operações trabalhem de forma integrada e automatizada. Essa integração é o núcleo da cultura DevOps, que combina práticas de integração e entrega contínuas, infraestrutura padronizada por containers e monitoramento constante da aplicação em produção (KIM et al., 2018; HUMBLE; FARLEY, 2014).

Este relatório documenta o projeto prático desenvolvido para a Atividade Final da disciplina DevOps Tools. O projeto abrange cinco etapas: (i) os conceitos fundamentais de DevOps; (ii) um pipeline de integração e entrega contínuas com GitHub Actions; (iii) a implantação contínua em um cluster Kubernetes por meio de Helm; (iv) a coleta de logs e métricas com Filebeat, Elasticsearch e Kibana; e (v) este relatório consolidado, com a reflexão crítica sobre o valor do DevOps.

Todo o código-fonte, o pipeline, o chart Helm e a configuração de monitoramento estão disponíveis no repositório público https://github.com/leao-leao/devops-atividade-final.

# 2 FUNDAMENTAÇÃO TEÓRICA

## 2.1 DevOps e sua relação com as metodologias ágeis

DevOps é uma cultura e um conjunto de práticas que integra as equipes de desenvolvimento (Dev) e de operações (Ops) para entregar software com mais rapidez, frequência e confiabilidade. Em vez de times isolados, que repassam o código de uma etapa à outra, desenvolvedores e operadores compartilham a responsabilidade pelo produto durante todo o seu ciclo de vida, apoiados por automação, infraestrutura como código, monitoramento e feedback contínuo (KIM et al., 2018).

As metodologias ágeis, como Scrum, Kanban e Extreme Programming, valorizam ciclos curtos de desenvolvimento, colaboração e resposta rápida a mudanças (BECK et al., 2001). Entretanto, de nada adianta a equipe produzir uma funcionalidade a cada iteração se ela leva semanas para chegar à produção. O DevOps estende a mentalidade ágil até a entrega e a operação: o ágil acelera o

desenvolvimento, e o DevOps acelera e estabiliza a entrega. Por analogia, o ágil é uma cozinha que prepara pratos em pequenas porções com rapidez, enquanto o DevOps é o processo que leva cada prato quente até a mesa, de forma padronizada e com avaliação constante dos clientes.

## 2.2 Integração contínua, entrega contínua e implantação contínua

A integração contínua (CI) consiste em integrar o código de todos os desenvolvedores várias vezes ao dia, compilando-o e testando-o automaticamente a cada alteração, para detectar erros o mais cedo possível. A entrega contínua (Continuous Delivery) amplia a CI: toda alteração aprovada nos testes gera um artefato pronto para ir à produção a qualquer momento, mas a publicação depende de uma aprovação humana. Na implantação contínua (Continuous Deployment), por fim, a publicação em produção também é automática (HUMBLE; FARLEY, 2014). A Tabela 1 resume as diferenças.

Tabela 1 – Diferenças entre CI, Continuous Delivery e Continuous Deployment

| **Prática** | **O que faz** | **Aprovação para produção** |
|---|---|---|
| Integração contínua (CI) | A cada alteração, o código é integrado, compilado e testado automaticamente. | Não chega à produção; apenas valida o código. |
| Continuous Delivery | Todo código aprovado gera um artefato pronto para implantação a qualquer momento. | Manual: uma pessoa aprova a publicação. |
| Continuous Deployment | Toda alteração aprovada nos testes é publicada em produção. | Automática: ninguém precisa aprovar. |

Fonte: Adaptado de Humble e Farley (2014).

No projeto, o job de CI do GitHub Actions executa os testes, constrói a imagem e a publica no registry (CI e artefato pronto para entrega), enquanto o job de deploy implanta a aplicação no Kubernetes automaticamente a cada alteração na ramificação principal, caracterizando a implantação contínua.

## 2.3 Máquinas virtuais e containers

Uma máquina virtual (VM) emula um computador completo, com sistema operacional próprio, sobre um hipervisor. Já os containers isolam apenas processos

e compartilham o núcleo (kernel) do sistema operacional hospedeiro, o que os torna mais leves e rápidos (DOCKER, 2026). A Tabela 2 compara as duas tecnologias.

Tabela 2 – Comparação entre máquinas virtuais e containers

| **Aspecto** | **Máquina virtual** | **Container** |
|---|---|---|
| Isolamento | Sistema operacional completo por VM, sobre hipervisor. | Processos isolados que compartilham o kernel do hospedeiro. |
| Tamanho | Gigabytes. | Megabytes (por exemplo, a imagem node:20-alpine). |
| Inicialização | Minutos. | Segundos ou menos. |
| Consumo de recursos | Alto, pois cada VM carrega um sistema operacional. | Baixo; comporta muito mais instâncias por servidor. |
| Portabilidade | Imagens pesadas e dependentes do hipervisor. | O mesmo artefato executa em qualquer ambiente com runtime de containers. |

Fonte: Adaptado de Docker (2026).

As principais vantagens dos containers são ambientes idênticos do desenvolvimento à produção, builds reproduzíveis, implantações rápidas, escalonamento horizontal simples e melhor aproveitamento do hardware. Por analogia, uma VM é uma casa inteira, com fundação, encanamento e energia próprios; os containers são apartamentos de um mesmo prédio, que compartilham a estrutura (o kernel), mas mantêm cada unidade isolada.

## 3 METODOLOGIA E FERRAMENTAS

O projeto foi desenvolvido em um repositório Git hospedado no GitHub. A Tabela 3 relaciona cada etapa da atividade às ferramentas utilizadas e aos artefatos produzidos.

Tabela 3 – Etapas, ferramentas e artefatos do projeto

| **Etapa** | **Ferramentas** | **Artefatos no repositório** |
|---|---|---|
| 2 – CI/CD | Node.js, Docker, GitHub Actions, GitHub Container Registry | app/, .github/workflows/main.yml |
| 3 – Orquestração | Kubernetes (Kind), Helm | helm/tarefas-api/ |
| 4 – Logs e métricas | Filebeat, Elasticsearch, Kibana, Docker Compose | elk/ |

Fonte: Elaborado pelos autores (2026).

# 4 DESENVOLVIMENTO

## 4.1 Aplicação

Foi desenvolvida uma API REST de tarefas em Node.js com o framework Express (diretório app/). Ela expõe os endpoints /health, /api/tarefas (GET, POST e DELETE), /api/lento e /api/erro; os dois últimos geram latência variável e erros HTTP 500, a fim de produzir dados relevantes para o monitoramento. A cada requisição, a aplicação escreve na saída padrão um log estruturado em JSON contendo método, caminho, status_code e response_time_ms. Cinco testes automatizados, escritos com o módulo nativo node:test, validam o funcionamento dos endpoints e do log.

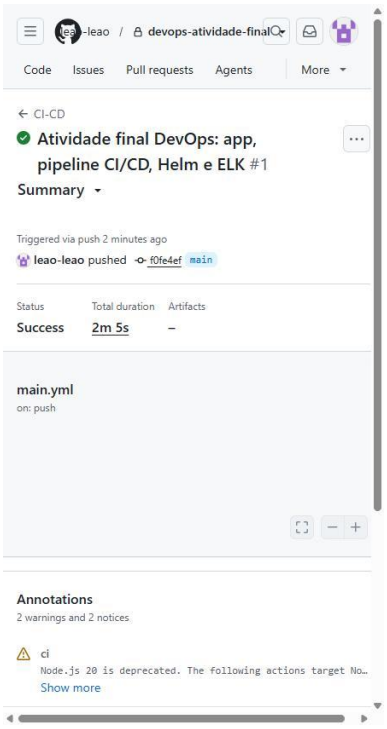
## 4.2 Integração e entrega contínuas (Etapa 2)

O pipeline está definido em .github/workflows/main.yml e é acionado a cada push ou pull request na ramificação principal (GITHUB, 2026). O job ci executa, em sequência:

a) instalação das dependências e execução dos testes automatizados (npm test);
b) construção da imagem Docker, a partir do Dockerfile baseado em node:20-alpine, com execução por usuário não administrador e verificação de saúde (HEALTHCHECK);
c) publicação da imagem no GitHub Container Registry (ghcr.io), com as etiquetas do hash do commit e latest, apenas em pushes na ramificação principal.

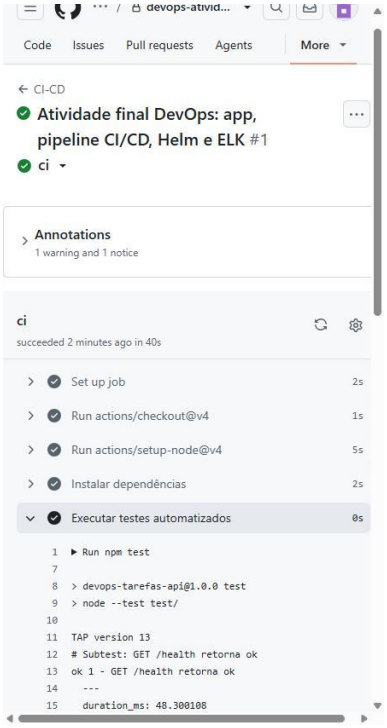
A Figura 1 mostra a execução completa do pipeline, com os jobs ci e deploy concluídos com sucesso em 2 minutos e 5 segundos; a Figura 2 mostra os cinco testes aprovados; e a Figura 3 mostra a imagem publicada no registry.

Figura 1 – Pipeline CI-CD executado com sucesso



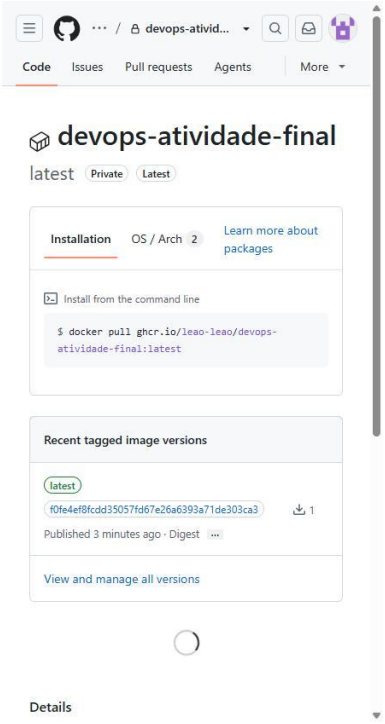

Fonte: Elaborado pelos autores (2026).

Figura 2 – Job ci: cinco testes aprovados




Fonte: Elaborado pelos autores (2026).

Figura 3 – Imagem publicada no ghcr.io




Fonte: Elaborado pelos autores (2026).

### 4.3 Implantação contínua e orquestração (Etapa 3)

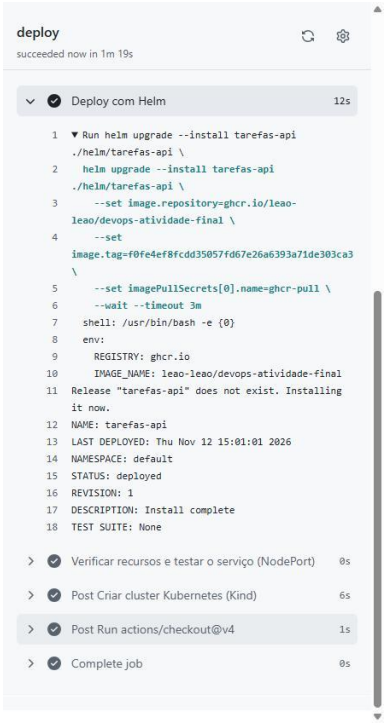
O diretório helm/tarefas-api contém o chart Helm (HELM, 2026) com os modelos de Deployment, Service e Ingress opcional. O Deployment executa duas réplicas com sondas de prontidão e de vivacidade em /health e limites de recursos; o Service é do tipo NodePort, na porta 30080 (KUBERNETES, 2026).

O job deploy do pipeline é executado após o CI: cria um cluster Kubernetes efêmero com Kind no próprio runner, registra um segredo para obter a imagem do registry e executa o comando helm upgrade --install, com a etiqueta da imagem igual ao hash do commit, de modo que cada alteração publica automaticamente uma nova versão. Em seguida, o job lista os recursos e testa o serviço pela porta NodePort. Para uso local, o mesmo chart pode ser instalado em um cluster Minikube ou Kind:

```
kind create cluster
helm upgrade --install tarefas-api ./helm/tarefas-api \
  --set image.repository=ghcr.io/leao-leao/devops-atividade-final
kubectl get pods,svc
```

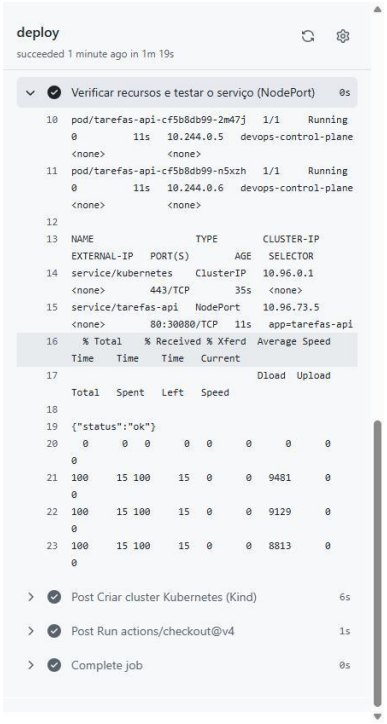
A Figura 4 registra a instalação do chart (status deployed) e a Figura 5 mostra os dois pods em execução (Running), o serviço NodePort 80:30080 e a resposta {"status":"ok"} do endpoint de saúde.

Figura 4 – Instalação do chart com Helm




Fonte: Elaborado pelos autores (2026).

Figura 5 – Pods e serviço no Kubernetes




Fonte: Elaborado pelos autores (2026).

## 4.4 Logging e métricas (Etapa 4)

A pasta elk/ contém um arquivo Docker Compose que sobe a aplicação, o Elasticsearch, o Kibana e o Filebeat, todos na versão 8.13.4 (ELASTIC, 2026). O processo de coleta dos dados pelo Filebeat ocorre da seguinte forma:

a) a aplicação escreve cada log em JSON na saída padrão, e o Docker, configurado com o driver json-file, grava essas linhas em /var/lib/docker/containers/<id>/<id>-json.log;

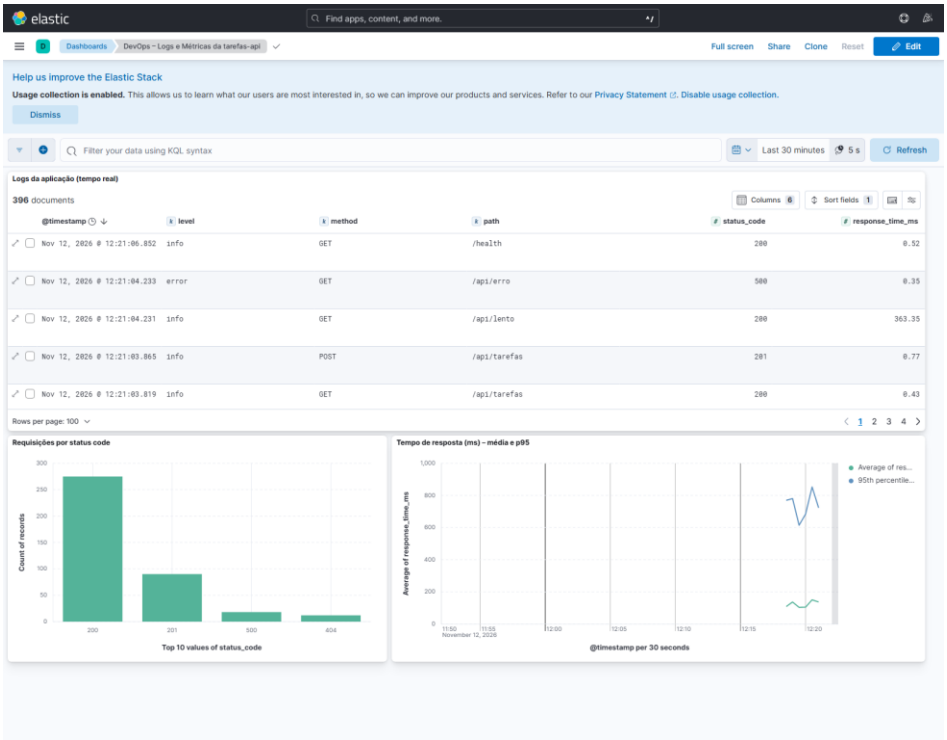
b) o Filebeat monta esse diretório em modo somente leitura e utiliza o input container para ler os arquivos; o processador add_docker_metadata identifica o container de origem e o processador drop_event descarta os registros dos demais containers (Elasticsearch, Kibana etc.);

c) o processador decode_json_fields converte o campo message, que contém o JSON em texto, em campos individuais (status_code, response_time_ms, method, path e level), que passam a ser numéricos e agregáveis;

d) o Filebeat envia os eventos ao Elasticsearch, no índice tarefas-api-logs-AAAA.MM.DD, que o Kibana consulta por meio de um Data View de padrão tarefas-api-logs-*.

No Kibana, foi criado um dashboard com três painéis: os logs da aplicação em tempo real (atualização a cada cinco segundos), a quantidade de requisições por status_code e o tempo de resposta ao longo do tempo (média e percentil 95). Os objetos do dashboard foram exportados para o arquivo elk/kibana-dashboard.ndjson, que pode ser importado em Stack Management > Saved Objects. A Figura 6 apresenta o resultado.

Figura 6 – Dashboard no Kibana com logs e métricas de requisições




Fonte: Elaborado pelos autores (2026).

## 5 RESULTADOS E DESAFIOS

O pipeline executou com sucesso do commit até a implantação: os testes foram aprovados, a imagem foi publicada no registry e o chart Helm foi instalado no cluster, com o serviço respondendo no endpoint de saúde. O monitoramento indexou as requisições da aplicação no Elasticsearch com campos numéricos, o que permitiu

construir os gráficos de status e de latência no Kibana. Os principais desafios enfrentados foram:

a) o nome da imagem no ghcr.io precisa estar em letras minúsculas, o que exigiu converter automaticamente o nome do repositório no workflow;
b) os runners do GitHub não possuem cluster permanente, razão pela qual se utilizou um cluster Kind efêmero no job de deploy;
c) as imagens do ghcr.io são privadas por padrão, o que exigiu a criação de um segredo de pull com o token do próprio workflow;
d) o autodiscover do Filebeat não encontrou o container e o Docker utilizava o driver de log local, que não gera arquivos *-json.log; a solução foi fixar o driver json-file no Compose e ler os arquivos com o input container;
e) o Elasticsearch rejeitava os eventos (erro 400), pois o campo service do log da aplicação, de tipo texto, conflitava com o campo service, de tipo objeto, do esquema ECS do Filebeat; o campo foi renomeado para app;
f) o Elasticsearch consome muita memória, o que levou a limitar o heap a 512 MB e a utilizar o modo de nó único.

## 6 CONSIDERAÇÕES FINAIS

A prática mostrou que a automação torna a entrega de software um processo repetível e previsível. O pipeline impede que qualquer alteração chegue ao cluster sem passar pelos testes; o versionamento da imagem pelo hash do commit permite rastrear exatamente o que está em execução e reverter uma versão com helm rollback; e o Helm padroniza a configuração da implantação. Com isso, reduzem-se os erros manuais e o tempo entre escrever e entregar o código cai de dias para minutos.

O monitoramento fecha o ciclo: logs estruturados e métricas de latência e de erro no Kibana fornecem feedback imediato sobre o comportamento real da aplicação, permitindo detectar regressões e gargalos antes que os usuários os percebam. Esse feedback realimenta o desenvolvimento, que é a essência da melhoria contínua defendida pelo DevOps e pelas metodologias ágeis (KIM et al., 2018).

Como limitações, destacam-se o esforço inicial de configuração, a necessidade de uma cultura colaborativa entre as equipes e os cuidados com segurança, como o gerenciamento de segredos, imagens e permissões, que também devem ser

automatizados. Como trabalhos futuros, sugerem-se a implantação em um cluster permanente, a adoção de alertas no Kibana e a inclusão de varredura de vulnerabilidades no pipeline.

## REFERÊNCIAS

BECK, K. *et al.* **Manifesto para desenvolvimento ágil de software**. 2001. Disponível em: https://agilemanifesto.org/iso/ptbr/manifesto.html. Acesso em: 12 nov. 2026.

DOCKER. **Docker documentation**. 2026. Disponível em: https://docs.docker.com. Acesso em: 12 nov. 2026.

ELASTIC. **Filebeat reference**. 2026. Disponível em: https://www.elastic.co/guide/en/beats/filebeat/current/index.html. Acesso em: 12 nov. 2026.

GITHUB. **GitHub Actions documentation**. 2026. Disponível em: https://docs.github.com/actions. Acesso em: 12 nov. 2026.

HELM. **Helm documentation**. 2026. Disponível em: https://helm.sh/docs. Acesso em: 12 nov. 2026.

HUMBLE, J.; FARLEY, D. **Entrega contínua**: como entregar software de forma rápida e confiável. Porto Alegre: Bookman, 2014.

KIM, G. *et al.* **Manual de DevOps**: como obter agilidade, confiabilidade e segurança em organizações tecnológicas. Rio de Janeiro: Alta Books, 2018.

KUBERNETES. **Kubernetes documentation**. 2026. Disponível em: https://kubernetes.io/docs. Acesso em: 12 nov. 2026.